

**LAPORAN PRAKTIKUM**  
**ALGORITMA DAN PEMROGRAMAN 2**  
**MODUL 12**  
**“SEARCHING”**



**DISUSUN OLEH:**  
**DAYANA RISTA NUR FAUZIAH**  
**109082500195**  
**S1 IF-13-02**  
**DOSEN:**

**Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2024/2025**

## **DASAR TEORI**

### **1. Sequential Search**

Sequential Search adalah metode pencarian data dengan cara memeriksa elemen satu per satu secara berurutan mulai dari data pertama hingga data yang dicari ditemukan atau seluruh data selesai diperiksa. Algoritma ini sederhana dan mudah digunakan, terutama untuk data yang belum terurut.

Cara kerja Sequential Search dilakukan dengan membandingkan setiap data dengan nilai yang dicari. Jika data ditemukan, maka proses berhenti. Jika sampai akhir data tidak ditemukan, maka pencarian dianggap gagal.

Kelebihan Sequential Search adalah mudah dibuat dan cocok untuk jumlah data sedikit. Kekurangannya adalah proses pencarian menjadi lebih lambat jika jumlah data sangat banyak.

### **2. Binary Search**

Binary Search adalah metode pencarian data pada kumpulan data yang sudah terurut. Algoritma ini bekerja dengan membagi data menjadi dua bagian, kemudian menentukan apakah data yang dicari berada di sebelah kiri atau kanan dari data tengah.

Proses pencarian dilakukan berulang kali hingga data ditemukan atau tidak ada lagi bagian data yang bisa diperiksa. Karena ruang pencarian selalu dibagi dua, Binary Search lebih cepat dibandingkan Sequential Search.

Kelebihan Binary Search adalah pencarian lebih efisien dan cepat pada data besar. Namun, algoritma ini hanya dapat digunakan jika data sudah terurut.

## SOAL 1

Source Code:

```
package main

import "fmt"

func main() {
    var suara [21]int
    var data int
    var masuk int
    var sah int

    for {
        fmt.Scan(&data) //uuntuk input data
        masuk++         // menghitung data yg masuk

        if data == 0 { //berhenti kalo ada 0
            break
        }

        if data >= 1 && data <= 20 { //cek suara yang sah
            suara[data]++
            sah++ // menghitung suara yg sah
        }
    }

    fmt.Println("Suara masuk:", masuk)
    fmt.Println("Suara sah:", sah)

    for data = 1; data <= 20; data++ {
        if suara[data] > 0 {
            fmt.Println(data, ":", suara[data])
        }
    }
}
```

Output:

The image shows a Go program in a text editor and its execution output in a terminal. The program is a simple voting system. It defines an array 'suara' of size 21 (index 0 to 20) to store votes for 20 candidates. The 'main' function uses a loop to read input 'data' until a 0 is entered. It checks if 'data' is between 1 and 20. If so, it increments the vote count for that candidate in the 'suara' array and also increments a 'masuk' counter. After the loop, it prints the total votes ('masuk') and the votes for each candidate (from index 1 to 20). A second window shows the program's output, displaying the name 'dayana rista nur fauziah' and NIM '109082500195'.

```
no1.go > main
1 package main
2
3 import "fmt"
4
5 func main() {
6     var suara [21]int
7     var data int
8     var masuk int
9     var sah int
10
11     for {
12         fmt.Scan(&data) //untuk input data
13         masuk++         // menghitung data yg masuk
14
15         if data == 0 { //berhenti kalo ada 0
16             break
17         }
18
19         if data >= 1 && data <= 20 { //cek suara yang sah
20             suara[data]++
21             sah++ // menghitung suara yg sah
22         }
23     }
24
25     fmt.Println("Suara masuk:", masuk)
26     fmt.Println("Suara sah:", sah)
27
28     for data = 1; data <= 20; data++ {
29         if suara[data] > 0 {
30             fmt.Println(data, ":", suara[data])
31         }
32     }
33 }
34
```

```
PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12> go run "c:\MATA KULIAH\AI
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 11
Suara sah: 8
1 : 1
2 : 1
3 : 2
7 : 1
18 : 1
19 : 2
PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12>
```

Penjelasan :

Kode di atas menjelaskan tentang data suara pemilu dengan menggunakan perulangan (for) terus menerus sekaligus menambah hitungan total semua data. Dimana input tidak akan berhenti sebelum ada masukan 0 (if data == 0). Setiap angka akan diperiksa kalau angka dengan rentang 1-20 maka angkanya sah sehingga penghitungan suaranya bertambah (sah++) lalu pada array akan bertambah 1 (suara[data]++). Setelah perulangan berhenti, program mengecek total suara sah. (for data = 1; data <= 20) ini untuk cek kandidat mana yang memiliki suara lebih dari nol.

## SOAL 2

### Source Code:

```
package main

import "fmt"

func main() {
    var suara [21]int
    var data, masuk, sah, ketua, wakil int

    for {
        fmt.Scan(&data) // input suara
        masuk++

        if data == 0 { // berhenti jika 0
            break
        }

        if data >= 1 && data <= 20 { // cek suara valid
            suara[data]++ // tambah suara
            sah++ //hitung suara sah
        }
    }

    // awal ketua dan wakil
    ketua = 1
    wakil = 2

    for data = 1; data <= 20; data++ { //cari ketua dan wakil

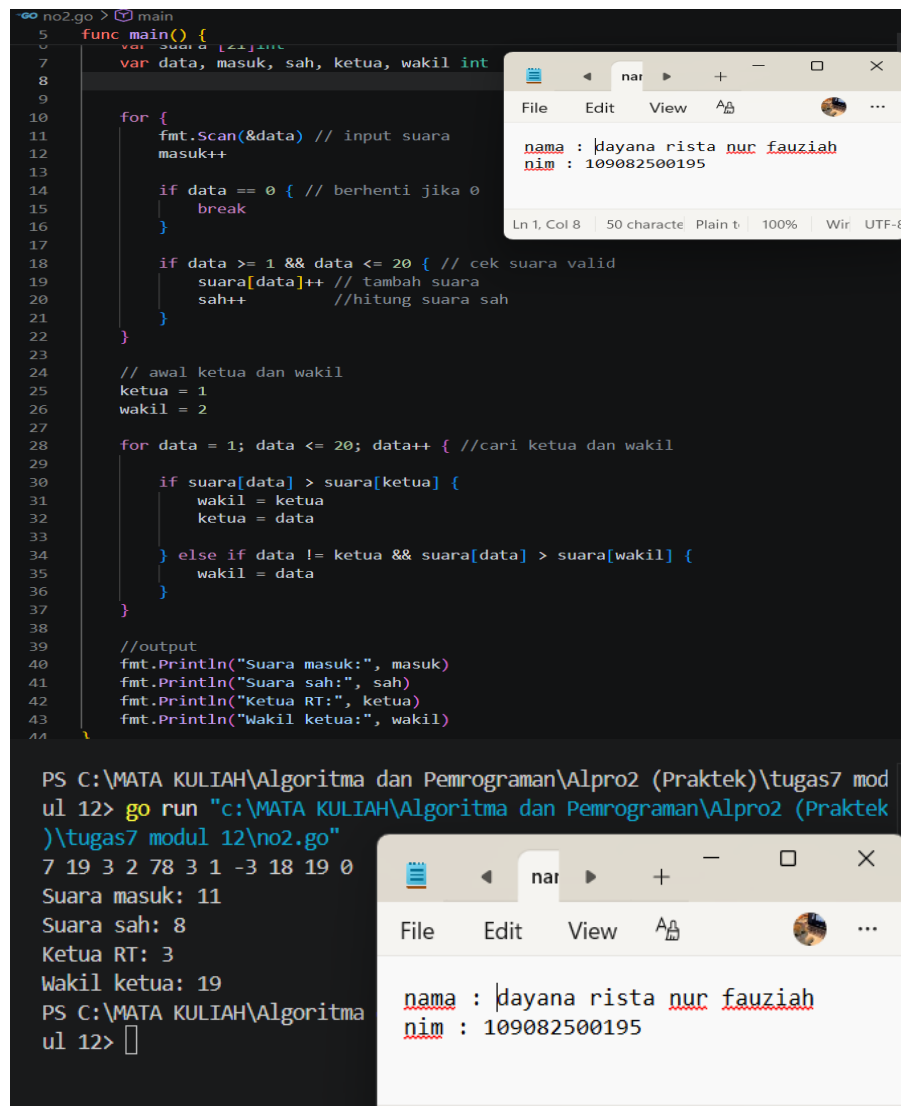
        if suara[data] > suara[ketua] {
            wakil = ketua
            ketua = data

        } else if data != ketua && suara[data] > suara[wakil] {
            wakil = data
        }
    }

    //output
    fmt.Println("Suara masuk:", masuk)
    fmt.Println("Suara sah:", sah)
    fmt.Println("Ketua RT:", ketua)
    fmt.Println("Wakil ketua:", wakil)
}

//dayan arista nur fauziah_109082500195
```

Output:



The screenshot displays a Go program in a text editor and its execution output in a terminal window. The program is a function `main()` that takes input from the user and calculates the RT Chairman and Deputy Chairman based on the number of votes.

```
5 func main() {
6     var suara [21]int
7     var data, masuk, sah, ketua, wakil int
8
9
10    for {
11        fmt.Scan(&data) // input suara
12        masuk++
13
14        if data == 0 { // berhenti jika 0
15            break
16        }
17
18        if data >= 1 && data <= 20 { // cek suara valid
19            suara[data]++ // tambah suara
20            sah++ //hitung suara sah
21        }
22    }
23
24    // awal ketua dan wakil
25    ketua = 1
26    wakil = 2
27
28    for data = 1; data <= 20; data++ { //cari ketua dan wakil
29
30        if suara[data] > suara[ketua] {
31            wakil = ketua
32            ketua = data
33        } else if data != ketua && suara[data] > suara[wakil] {
34            wakil = data
35        }
36    }
37
38    //output
39    fmt.Println("Suara masuk:", masuk)
40    fmt.Println("Suara sah:", sah)
41    fmt.Println("Ketua RT:", ketua)
42    fmt.Println("Wakil ketua:", wakil)
43
44 }
```

The terminal output shows the program execution results:

```
PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12> go run "c:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12\n02.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 11
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12>
```

Two windows are also visible in the background, both titled 'nar', showing the same text: `nama : dayana rista nur fauziah` and `nim : 109082500195`.

## Penjelasan :

Kode di atas menjelaskan tentang sistem penghitungan suara pemilihan ketua RT dengan menggunakan perulangan terus-menerus untuk membaca input angka. Program akan terus menambah hitungan total semua data yang masuk (`masuk++`) dan proses ini hanya akan berhenti jika pengguna memasukkan angka 0 (`if data == 0`). Setiap angka akan diperiksa; jika angka berada dalam rentang valid 1-20 (`if data >= 1 && data <= 20`), maka suara tersebut dianggap sah (`sah++`) dan jumlah suara untuk nomor kandidat tersebut di dalam array akan bertambah (`suara[data]++`). Setelah data selesai diinput, program menentukan Ketua dan Wakil Ketua melalui perulangan untuk membandingkan jumlah suara setiap kandidat. Jika ditemukan kandidat dengan suara yang lebih banyak dari Ketua saat ini, maka posisi Ketua akan digantikan dan Ketua lama turun menjadi Wakil. Jika suara kandidat hanya lebih besar dari Wakil, maka ia cukup menggantikan posisi Wakil Ketua saja. Di akhir, program

mencetak total suara masuk, jumlah suara sah, serta nomor kandidat yang terpilih sebagai Ketua dan Wakil Ketua.

### SOAL 3

#### Source Code:

```
package main

import "fmt"

const batas = 1000000

var data [batas]int

func isi(jumlah int) {
    var i int

    for i = 0; i < jumlah; i++ {
        fmt.Scan(&data[i]) // input array
    }
}

func cari(jumlah, angka int) int {
    var kiri int
    var kanan int
    var tengah int

    kanan = jumlah - 1 //posisi kanan

    for kiri <= kanan {

        tengah = (kiri + kanan) / 2 //posisi kanan

        if data[tengah] == angka { // data ditemukan
            return tengah

        } else if data[tengah] < angka { // geser kanan
            kiri = tengah + 1

        } else {
            kanan = tengah - 1 // geser kiri
        }
    }

    return -1 // tidak ditemukan
}

func main() {
    var jumlah, angka, hasil int
```

```

fmt.Scan(&jumlah, &angka) // input jumlah dan angka
isi(jumlah)               // isi array
hasil = cari(jumlah, angka) // cari data

// output
if hasil == -1 {
    fmt.Println("TIDAK ADA")
} else {
    fmt.Println(hasil)
}
}

```

Output:

The screenshot shows a Go code editor with a dark theme. The code implements a binary search algorithm. A function `cari` takes `jumlah` (array size) and `angka` (target value) as arguments and returns the index of the target value or -1 if not found. The `main` function uses `fmt.Scan` to read input, `isi` to initialize the array, and `cari` to search for the target. The output window shows the search results for the name 'Dayana rista nur fauziah' and NIM '109082500195'.

```

15 }
16
17 func cari(jumlah, angka int) int {
18     var kiri int
19     var kanan int
20     var tengah int
21
22     kanan = jumlah - 1 //posisi kanan
23
24     for kiri <= kanan {
25
26         tengah = (kiri + kanan) / 2 //posisi kanan
27
28         if data[tengah] == angka { // data ditemukan
29             return tengah
30         } else if data[tengah] < angka { // geser kanan
31             kiri = tengah + 1
32         } else {
33             kanan = tengah - 1 // geser kiri
34         }
35     }
36     return -1 // tidak ditemukan
37 }
38
39 }
40
41 func main() {
42     var jumlah, angka, hasil int
43
44     fmt.Scan(&jumlah, &angka) // input jumlah dan angka
45     isi(jumlah) // isi array
46     hasil = cari(jumlah, angka) // cari data
47
48     // output
49     if hasil == -1 {
50         fmt.Println("TIDAK ADA")
51     } else {
52         fmt.Println(hasil)
53     }
54 }

```

Output window:

```

nama : Dayana rista nur fauziah
nim  : 109082500195

```

Ln 1, Col 8 | 50 character | Plain text | 100% | Windows | UTF-8



```
nama : dayana rista nur fauziah
nim : 109082500195

PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12> go run "c:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12\n03.go"
12 534
1 3 8 16 32 123 323 323 534 343 823 999
8
PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12> go run "c:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12\n03.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS C:\MATA KULIAH\Algoritma dan Pemrograman\Alpro2 (Praktek)\tugas7 modul 12>
```

Penjelasan :

Kode ini menjelaskan tentang algoritma Binary Search untuk mencari posisi suatu angka dalam kumpulan data yang sudah terurut membesar. Program mulai dengan membaca jumlah data dan angka yang ingin dicari (`fmt.Scan(&jumlah, &angka)`), lalu mengisi array melalui prosedur isi. Inti dari kode ini ada pada fungsi `cari`, di mana pencarian dilakukan dengan membagi rentang data menjadi dua secara terus-menerus melalui titik tengah ( $\text{tengah} = (\text{kiri} + \text{kanan}) / 2$ ). Jika angka di titik tengah sama dengan yang dicari, program langsung mengembalikan indeksinya. Namun, jika angka tengah terlalu kecil, rentang pencarian digeser ke kanan ( $\text{kiri} = \text{tengah} + 1$ ), dan jika terlalu besar, rentang digeser ke kiri ( $\text{kanan} = \text{tengah} - 1$ ). Proses ini diulang selama posisi kiri belum melewati kanan; jika sampai akhir tidak ditemukan, program mengembalikan nilai -1 yang akan mencetak output "TIDAK ADA", atau mencetak indeks angka tersebut jika ditemukan.

## DAFTAR PUSTAKA

<https://dosenit.com/ilmu-komputer/binary-search>

<https://fikti.umsu.ac.id/algorithm-sequential-search-pengertian-fungsi-dan-cara-kerjanya/>